COMPUTER ENGINEERING

QUEZON CITY

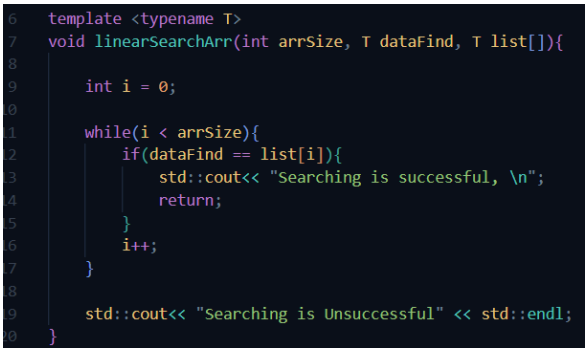
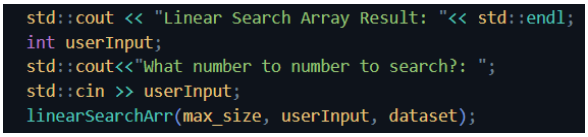
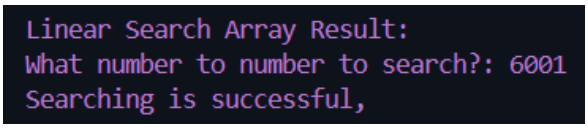
COURSE CODE/TITLE:	CPE010	**SECTION:**	CpE21S3
DATE PERFORMED:	08/26/2026	**DATE SUBMITTED:**	08/26/2026
NAME:	Wayne Kaisen C. Paraiso	**INSTRUCTOR:**	Engr. Jimjord Quejado

Hands-on Activity 6.1 Searching Techniques

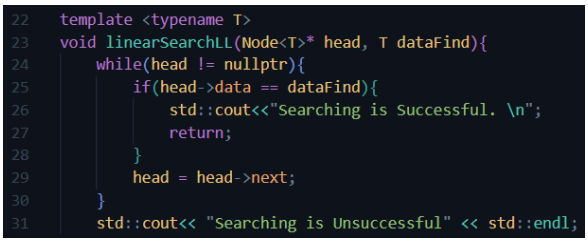
1. PROCEDURE OUTPUT

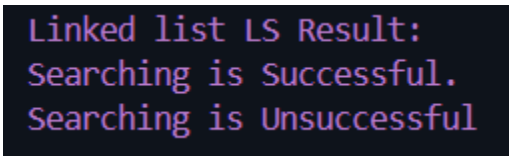
Screenshot	
Observation	The above output is the set of 50 random numbers that have been generated using the dataset array. They are displayed in the same order in which they have been generated and not in ascending or descending order, which is clear from the fact that there are no numbers in any particular numerical order because they are direct outputs of the rand() function without any sorting being done; therefore, they are in the “before” stage.

6.1 Data Generated and Observations

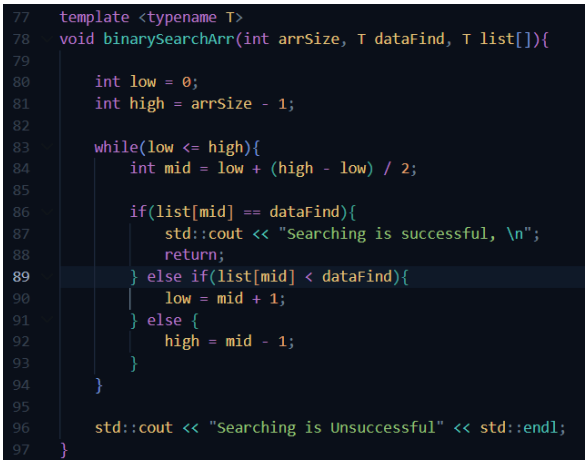
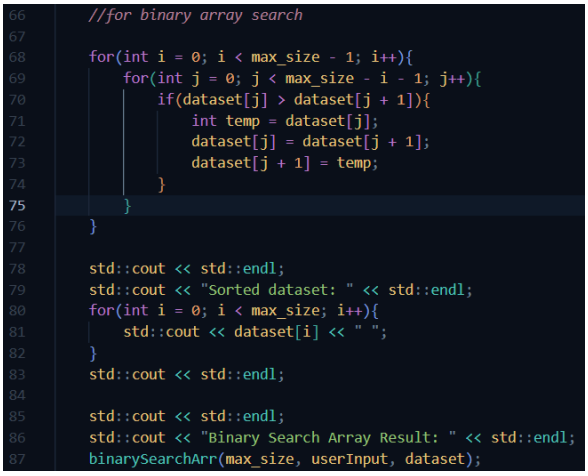
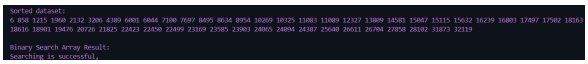
Code	 
Output	
Observations	The purpose of the function is to carry out a linear search on the array, where the function takes the size of the array, the value to search for, and the array as inputs, after which it will traverse the array by incrementing the counter from zero and check on each traversal whether the element in the array equals the value to be searched for; when the element in the array equals the value to be searched for, it will print the success message and terminate the function, but in case of failure, the function will print the failed message.

6.2 Linear Search for Arrays

Code	

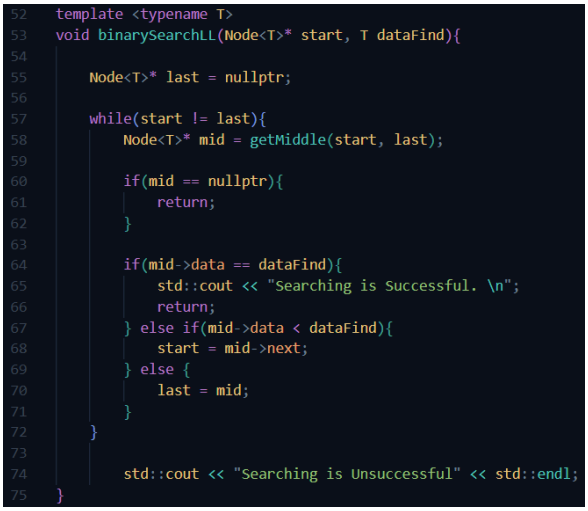
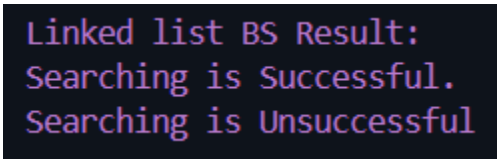
	
Output	
Observations	In this function, we will use the linear search technique to search an element in the singly linked list by passing two arguments: the pointer to the head node and the element that needs to be searched, and in the while loop, it moves through the linked list starting from the head node to the next node by accessing the next pointer of the node, until the pointer becomes null (end of list), and in every iteration, it checks whether the element stored in that particular node is equal to the element which we need to search, and if found, then print a success message and exit the function.

6.3 Linear Search for Linked List

Code	 
Output	
Observations	The program executes a binary search algorithm for a sorted array by using the size of the array, the value that is required to be searched in the array, and the array as the input parameters, initializing two markers for the lowest and highest indices of the array; until the lowest marker surpasses the highest marker, the program keeps calculating the middle index between the two markers, compares the element at the middle index with the required element; if it finds that the middle index contains the element being searched, it immediately displays a success message and exits from the function, but if the middle element is less than the required element, it moves the lowest index marker above the middle index, or if the middle element is greater, it moves the highest index marker below the middle index; when the markers cross each

	other without finding any match, the program stops the loop and displays an unsuccessful message.

6.4 Binary Search for Arrays

Code	 
Output	
Observations	In this method, binary search is performed on a sorted singly linked list using a pointer to the starting node and the search value as the parameters and initializes a last pointer to nullptr to indicate the boundary at the end of the search window. While the start pointer is yet to reach the boundary at the last, it repeatedly invokes the helper method getMiddle method to get the node

	in the middle of the start and last and breaks out of the loop if no such middle node exists; otherwise, it verifies whether the data of the middle node is equal to the search value, and if so, outputs that the search was successful and breaks out of the loop; otherwise, it compares whether the data in the middle node is less than the search value, and if so, updates the start pointer to a position after the middle node, narrowing down the search to the latter half of the list, otherwise, the last pointer is updated to the middle node, narrowing down the search to the former half of the list.

6.5 Binary Search for Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1

Code

```
int linearSearchArr(int arrSize, T dataFind, T list[]){

    int comparisons = 0;
    int i = 0;

    while(i < arrSize){
        comparisons++;
        if(dataFind == list[i]){
            std::cout << "Searching is successful. Found '" << dataFind
                      << "' at index " << i << " using "
                      << comparisons << " comparison(s).\n";
            return comparisons;
        }
        i++;
    }

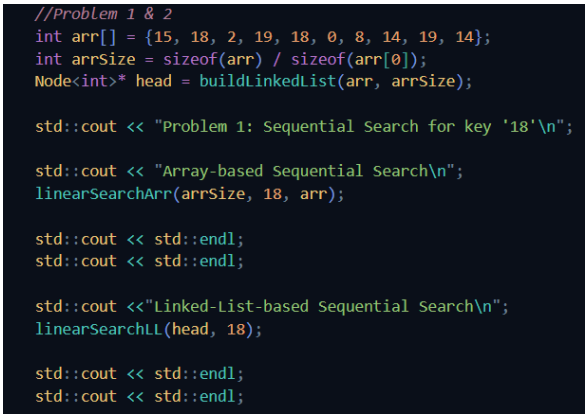
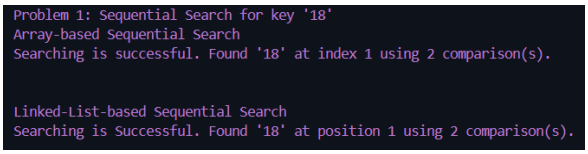
    std::cout << "Searching is Unsuccessful after "
              << comparisons << " comparison(s)." << std::endl;
    return comparisons;
}
```

```
template <typename T>
int linearSearchLL(Node<T>* head, T dataFind){

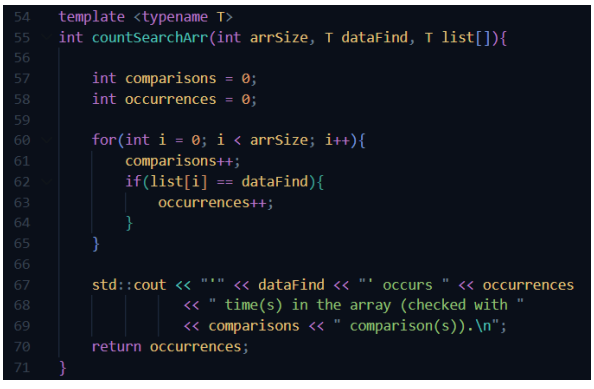
    int comparisons = 0;
    int position = 0;

    while(head != nullptr){
        comparisons++;
        if(head->data == dataFind){
            std::cout << "Searching is Successful. Found '" << dataFind
                      << "' at position " << position << " using "
                      << comparisons << " comparison(s).\n";
            return comparisons;
        }
        head = head->next;
        position++;
    }

    std::cout << "Searching is Unsuccessful after "
              << comparisons << " comparison(s)." << std::endl;
    return comparisons;
}
```

	
Output	
Answer	The algorithm scans the list element by element starting from the beginning of the list and compares each element to the target element while increasing the counter after each comparison so that it terminates immediately when it comes across a matching element – in this case, it will take two comparisons (15 and 18) regardless of the storage structure of the list being array or linked list because sequential search doesn't depend on the underlying data structure.

Problem 2

Code	

```cpp
template <typename T>
int countSearchLL(Node<T>* head, T dataFind){

    int comparisons = 0;
    int occurrences = 0;

    while(head != nullptr){
        comparisons++;
        if(head->data == dataFind){
            occurrences++;
        }
        head = head->next;
    }

    std::cout << "'" << dataFind << "' occurs " << occurrences
              << " time(s) in the linked list (checked with "
              << comparisons << " comparison(s)).\n";
    return occurrences;
}
```

```cpp
template <typename T>
Node<T>* getMiddle(Node<T>* start, Node<T>* last){

    Node<T>* slow = start;
    Node<T>* fast = start;

    while(fast != last && fast->next != last){
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow;
}
```

```cpp
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind){

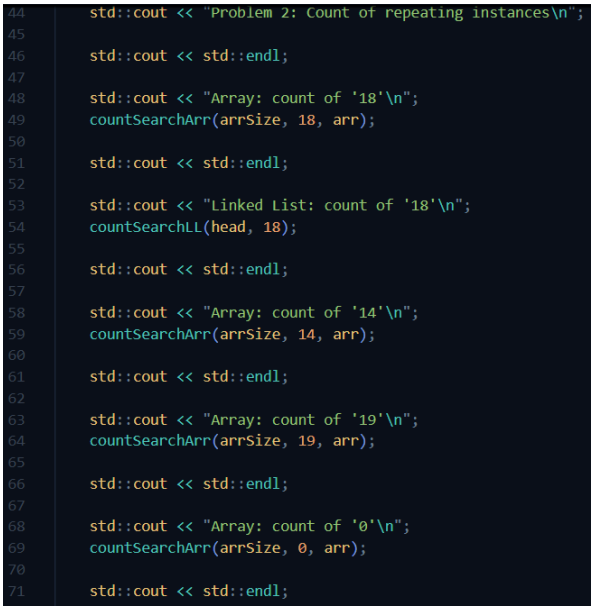
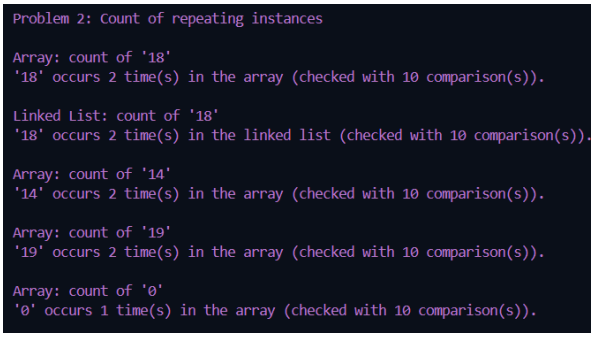
    Node<T>* last = nullptr;

    while(start != last){
        Node<T>* mid = getMiddle(start, last);

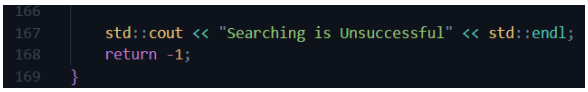
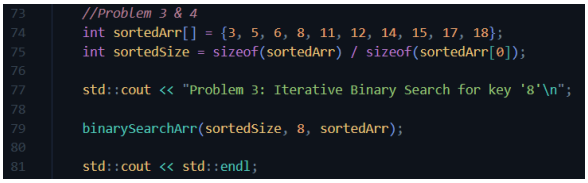
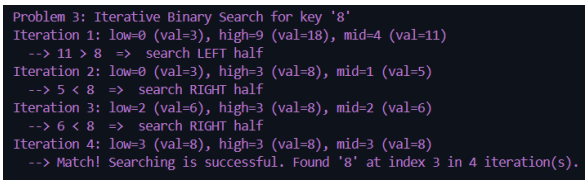
        if(mid == nullptr){
            break;
        }

        if(mid->data == dataFind){
            std::cout << "Searching is Successful. \n";
            return;
        } else if(mid->data < dataFind){
            start = mid->next;
        } else {
            last = mid;
        }
    }

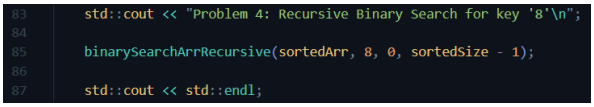
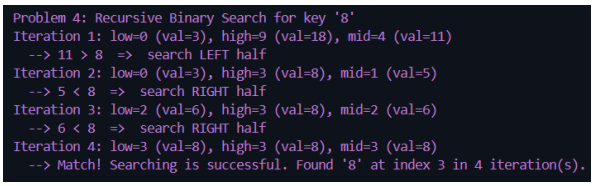
    std::cout << "Searching is Unsuccessful" << std::endl;
}
```

	
Output	
Answer	In contrast to the conventional method, which stops after finding the first match, the enhanced approach examines every item in the list, one time each, and counts the number of occurrences of that particular item. Thus, it provides not only the number of comparisons made but also the actual number of times the particular item occurs, like '18', which occurs twice. The reason why sequential search has such an approach is because it has no mechanism to "jump ahead."

Problem 3

Code	  
Output	
Answer	Since the array is sorted, the algorithm continually checks the middle element of the current range and exploits the knowledge that all elements to the left of it are lesser and all elements to the right of it are greater to discard half of the list on every iteration – checking whether 8 is greater or lesser than the middle number, and accordingly shifting the boundaries leftward or rightward, till the middle element becomes equal to 8 (in 4 iterations), which is much less than what sequential search will take due to reduction in problem size by half in every step.

Problem 4

Code	 
Output	
Answer	The difference here is that although this version contains the exact same logic of "compare against the middle, reduce the search range," it does not contain the loop; each time through the range is reduced until the base case (either finding the matching number or having an invalid range where low > high) is found, which results in the exact same set of iterations as the previous example and shows that recursion is simply a refactoring of the same idea of divide and conquer.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction, summarization**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I have a draft of the answers but I let claude correct my grammar and summarize what I wrote**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.